

Integrated Mobile Robot Navigation with Reactive Obstacle Avoidance and Filtered Range Sensors

Julio Quejido Barrera
Humanoide Robotik
Julio's Laboratories

July 29, 2026

Abstract

This project integrates goal-seeking control, finite range-sensor rays, geometric obstacle detection, reactive obstacle avoidance, Gaussian measurement noise, and exponential sensor filtering in a single mobile robot simulation. In free space, the robot follows a goal using a proportional angular controller and a distance-dependent linear-speed law with saturation. Three range sensors, oriented to the left, right, and front of the robot, test finite rays against circular obstacles. When an obstacle is detected, a priority-based reactive controller temporarily replaces the goal-seeking behavior. The ideal ray-circle intersection distance is then treated as a sensor measurement, corrupted with additive Gaussian noise, and smoothed using a first-order exponential moving-average filter. The resulting system represents an integrated sensing-decision-motion pipeline in which continuous feedback control is combined with discrete reactive rules and basic signal processing.

1 Introduction

A mobile robot that moves toward a target must continuously solve three related problems. First, it must calculate how fast to move and how much to rotate in order to approach the goal. Second, it must perceive nearby obstacles and determine whether its current motion is safe. Third, it must deal with imperfect sensor measurements, because real distance sensors do not return exact geometric values.

The present project combines these problems in one simulation. The vector and angular operations developed in the previous projects are reused to calculate the current distance and normalized angular error. These quantities are now converted into linear and angular velocities. At the same time, three finite sensor rays are generated from the robot pose and tested against a set of circular obstacles. The closest valid intersection detected by each sensor is converted into a binary left-right-front perception state.

The navigation behavior is selected by priority. If no sensor detects an obstacle, the robot follows the goal with proportional control. If an obstacle is detected, the goal controller is temporarily replaced by a reactive

avoidance rule. Finally, Gaussian noise and exponential smoothing are introduced so that the control decision is based on a filtered measurement rather than on an ideal intersection distance. This extends the simulation from pure geometry to an integrated model of control, perception, uncertainty, and filtering.

2 Theoretical Background

2.1 Distance to the Goal and Stopping Criterion

Let the robot position at simulation step k be $\mathbf{p}_k = [x_k, y_k]^T$ and let the fixed goal position be $\mathbf{p}_G$. The Euclidean distance, already introduced in the first project, is reused as the main translational feedback variable:

$$d_k = \|\mathbf{p}_G - \mathbf{p}_k\|. \quad (1)$$

The simulation terminates when the robot enters a circular tolerance region around the goal:

$$d_k < d_{\text{tol}}, \quad d_{\text{tol}} = 0.01. \quad (2)$$

The distance is therefore not rederived here as a geometric concept; it is used as a feedback signal, a stopping condition, and an input to the linear-speed controller.

2.2 Normalized Angular Error

The robot-to-goal vector is

$$\mathbf{v}_{G,k} = \mathbf{p}_G - \mathbf{p}_k.$$

Its absolute direction is calculated with the two-argument arctangent and converted to degrees:

$$\theta_{G,k}^\circ = \frac{180^\circ}{\pi} \text{atan2}(v_{G,y,k}, v_{G,x,k}). \quad (3)$$

The angular error reused by the controller is the shortest signed difference between the goal direction and the current robot orientation:

$$e_{\theta,k} = (\theta_{G,k}^\circ - \theta_k^\circ + 180^\circ) \bmod 360^\circ - 180^\circ. \quad (4)$$

Consequently,

$$-180^\circ \leq e_{\theta,k} < 180^\circ.$$

Angle normalization was developed in the previous project. Its new role here is to provide a continuous control error whose sign specifies the shortest turning direction and whose magnitude specifies the required orientation correction.

2.3 Angular Proportional Controller

In free space, the angular velocity is generated by a proportional controller:

$$\omega_k = K_\theta e_{\theta,k}. \quad (5)$$

The implemented angular gain is

$$K_\theta = 0.5.$$

Because $e_{\theta,k}$ is represented in degrees, ω_k is treated as a rotational rate in degrees per unit time. A large orientation error produces a large turning command, while a small error produces a slower correction. When the robot is aligned with the goal, $e_{\theta,k} = 0$ and the proportional command becomes $\omega_k = 0$.

This equation marks the transition from angular analysis to closed-loop motion control. In the previous project, the error only indicated left, right, or straight. Here, the same error directly determines the continuous turning rate.

2.4 Linear-Speed Control and Saturation

When the path is free, the desired linear speed is proportional to the distance from the goal:

$$v_k^{\text{raw}} = K_v d_k. \quad (6)$$

A maximum-speed limit is applied to avoid commanding a value above the allowed motor speed:

$$v_k = \min(K_v d_k, v_{\max}). \quad (7)$$

Equivalently,

$$v_k = \begin{cases} K_v d_k, & K_v d_k < v_{\max}, \\ v_{\max}, & K_v d_k \geq v_{\max}. \end{cases} \quad (8)$$

The initial values used by the simulation are

$$K_v = 0.5, \quad v_{\max} = 1.0.$$

Far from the goal, the command reaches the saturation limit. As the robot approaches the target, the proportional term decreases and the robot slows down progressively.

2.5 Near-Goal Parameter Adjustment

Inside a distance of one unit from the goal, the simulation reduces both the linear gain and the sensor range:

$$K_v(d_k) = \begin{cases} 0.5, & d_k \geq 1, \\ 0.1, & d_k < 1, \end{cases} \quad (9)$$

and

$$R_s(d_k) = \begin{cases} 0.5, & d_k \geq 1, \\ 0.3, & d_k < 1. \end{cases} \quad (10)$$

This is a simple form of parameter scheduling. The smaller linear gain reduces the approach speed near the target, while the shorter sensor range makes the local avoidance region more compact. In the implemented simulation these values are assigned once the condition $d_k < 1$ is satisfied and are not explicitly reset later.

2.6 Discrete Robot Motion Model

The orientation angle is converted into a unit heading vector. Since the stored orientation is expressed in degrees, it is first converted to radians:

$$\theta_k^{\text{rad}} = \frac{\pi}{180^\circ} \theta_k^\circ. \quad (11)$$

The corresponding heading is

$$\mathbf{h}_k = \begin{bmatrix} \cos \theta_k^{\text{rad}} \\ \sin \theta_k^{\text{rad}} \end{bmatrix}. \quad (12)$$

The orientation update is

$$\theta_{k+1}^\circ = \theta_k^\circ + \omega_k \Delta t. \quad (13)$$

After updating the orientation, the new heading is used to advance the position:

$$\mathbf{p}_{k+1} = \mathbf{p}_k + v_k \mathbf{h}_{k+1} \Delta t. \quad (14)$$

In coordinate form,

$$x_{k+1} = x_k + v_k \cos \theta_{k+1}^{\text{rad}} \Delta t, \quad (15)$$

$$y_{k+1} = y_k + v_k \sin \theta_{k+1}^{\text{rad}} \Delta t. \quad (16)$$

The time step is $\Delta t = 0.01$. This discrete update is a simple unicycle-like kinematic model in which the control inputs are the linear speed v_k and angular speed ω_k .

2.7 Finite Range-Sensor Rays

The robot carries three sensors with fixed relative orientations:

$$\beta_L = 45^\circ, \quad \beta_R = -45^\circ, \quad \beta_F = 0^\circ. \quad (17)$$

For sensor $s \in \{L, R, F\}$, the global sensor angle is obtained by adding the relative sensor orientation to the robot orientation:

$$\phi_{s,k} = \theta_k + \beta_s. \quad (18)$$

The sensor direction is the unit vector

$$\mathbf{u}_{s,k} = \begin{bmatrix} \cos \phi_{s,k}^{\text{rad}} \\ \sin \phi_{s,k}^{\text{rad}} \end{bmatrix}. \quad (19)$$

A finite sensor ray is represented parametrically as

$$\mathbf{r}_{s,k}(t) = \mathbf{p}_k + t\mathbf{u}_{s,k}, \quad 0 \leq t \leq R_s. \quad (20)$$

Its endpoint is therefore

$$\mathbf{p}_{s,k}^{\text{end}} = \mathbf{p}_k + R_s \mathbf{u}_{s,k}. \quad (21)$$

Unlike an infinite mathematical line, this model only detects objects inside the interval from the robot position to the maximum range R_s .

2.8 Circular Obstacle Model

Each obstacle j is represented by a center $\mathbf{c}_j = [c_{x,j}, c_{y,j}]^T$ and a radius r_j . Its boundary is

$$\|\mathbf{p} - \mathbf{c}_j\| = r_j. \quad (22)$$

For a given robot position, the vector from the robot to the obstacle center is

$$\mathbf{q}_{j,k} = \mathbf{c}_j - \mathbf{p}_k. \quad (23)$$

The simulation tests every sensor ray against every circular obstacle. The geometric procedure first determines whether an intersection is possible and then calculates the first point at which the ray enters the circle.

2.9 Projection onto the Sensor Direction

The scalar projection of the obstacle-center vector onto the unit sensor direction is

$$\lambda_{s,j,k} = \mathbf{q}_{j,k} \cdot \mathbf{u}_{s,k}. \quad (24)$$

Because $\mathbf{u}_{s,k}$ is normalized, $\lambda_{s,j,k}$ is the longitudinal distance from the robot to the orthogonal projection of the obstacle center on the sensor line.

Before performing the complete intersection calculation, the obstacle is rejected when its projected interval does not overlap the finite ray:

$$\lambda_{s,j,k} + r_j < 0 \quad \text{or} \quad \lambda_{s,j,k} - r_j > R_s. \quad (25)$$

This inexpensive test excludes circles located completely behind the sensor or completely beyond its maximum range.

2.10 Closest Point and Perpendicular Distance

The point on the sensor line closest to the obstacle center is

$$\mathbf{p}_{s,j,k}^{\text{close}} = \mathbf{p}_k + \lambda_{s,j,k} \mathbf{u}_{s,k}. \quad (26)$$

The perpendicular distance from the obstacle center to the sensor line is

$$d_{s,j,k}^\perp = \|\mathbf{c}_j - \mathbf{p}_{s,j,k}^{\text{close}}\|. \quad (27)$$

An intersection with the infinite sensor line is possible only if

$$d_{s,j,k}^\perp \leq r_j. \quad (28)$$

If $d_{s,j,k}^\perp > r_j$, the ray passes outside the circle and the obstacle is ignored.

2.11 Ray–Circle Intersection Distance

When the line intersects the circle, the projection, radius, and perpendicular distance form a right triangle. The longitudinal offset from the projected center to either intersection is

$$\delta_{s,j,k} = \sqrt{r_j^2 - (d_{s,j,k}^\perp)^2}. \quad (29)$$

The two line–circle intersection parameters are

$$t_{s,j,k}^{\text{near}} = \lambda_{s,j,k} - \delta_{s,j,k}, \quad (30)$$

and

$$t_{s,j,k}^{\text{far}} = \lambda_{s,j,k} + \delta_{s,j,k}. \quad (31)$$

The simulation uses the near intersection because it corresponds to the first obstacle surface reached by the ray. A geometric hit is accepted only if

$$0 \leq t_{s,j,k}^{\text{near}} \leq R_s. \quad (32)$$

Without measurement noise, the ideal hit point would be

$$\mathbf{p}_{s,j,k}^{\text{hit}} = \mathbf{p}_k + t_{s,j,k}^{\text{near}} \mathbf{u}_{s,k}. \quad (33)$$

2.12 Nearest Detection per Sensor

A single sensor ray may intersect more than one obstacle. Let $\mathcal{I}_{s,k}$ denote the set of valid detected obstacles for sensor s at step k . The selected obstacle is the one with the smallest accepted measurement distance:

$$j_{s,k}^* = \arg \min_{j \in \mathcal{I}_{s,k}} d_{s,j,k}^{\text{meas}}. \quad (34)$$

Each sensor therefore returns either no detection or one record containing the nearest valid obstacle, its measured distance, the hit point, and the associated geometric closest point.

2.13 Binary Sensor State

The three selected sensor outputs are converted into binary states:

$$D_L, D_R, D_F \in \{0, 1\}. \quad (35)$$

A value of one means that the corresponding left, right, or front sensor contains a valid detection. These binary variables remove the detailed geometric information from the final avoidance decision and allow the robot to apply a compact rule-based controller.

2.14 Reactive Obstacle Avoidance

The avoidance behavior directly assigns linear and angular velocities from the binary sensor state. The implemented rules are summarized in Table 1. Positive angular velocity represents a left turn, while negative angular velocity represents a right turn.

Table 1: Priority-Ordered Reactive Avoidance Rules.

Priority	Detection	v_k	ω_k	Response
1	$D_L \wedge D_F$	0.25	-90	turn right
2	$D_R \wedge D_F$	0.25	+90	turn left
3	$D_R \wedge D_L$	0.10	0	move slowly forward
4	D_F	0.10	+50	turn left
5	D_L	0.50	-70	turn right
6	D_R	0.50	+70	turn left

The ordering is part of the controller. The conditions are evaluated sequentially, and only the first valid rule is executed. For example, if all three sensors detect simultaneously, the first condition $D_L \wedge D_F$ is satisfied and the robot applies the right-turn rule. The controller is reactive because it maps the current sensor state directly to an action without predicting future motion.

2.15 Behavior Arbitration

The system combines two behaviors: obstacle avoidance and goal seeking. Let the control vector be

$$\mathbf{u}_k = \begin{bmatrix} v_k \\ \omega_k \end{bmatrix}. \quad (36)$$

The arbitration rule is

$$\mathbf{u}_k = \begin{cases} \mathbf{u}_k^{\text{avoid}}, & D_L \vee D_R \vee D_F, \\ \mathbf{u}_k^{\text{goal}}, & \neg(D_L \vee D_R \vee D_F), \end{cases} \quad (37)$$

where the free-space goal controller is

$$\mathbf{u}_k^{\text{goal}} = \begin{bmatrix} \min(K_v d_k, v_{\max}) \\ K_{\theta} e_{\theta,k} \end{bmatrix}. \quad (38)$$

Obstacle avoidance has strict priority over goal seeking. The two commands are not averaged or blended. A detection switches the system to the reactive mode; when all sensors are free, the proportional goal controller becomes active again. The complete system is therefore a simple hybrid or switched controller that combines discrete decisions with continuous robot motion.

2.16 Additive Gaussian Sensor Noise

The geometric near-intersection distance is treated as the ideal sensor value:

$$d_{s,j,k}^{\text{true}} = t_{s,j,k}^{\text{near}}. \quad (39)$$

A noisy measurement is generated using an additive Gaussian model:

$$z_n = d_{s,j,k}^{\text{true}} + \eta_n, \quad (40)$$

with

$$\eta_n \sim \mathcal{N}(0, \sigma^2). \quad (41)$$

The probability density of the noise is

$$p(\eta) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{\eta^2}{2\sigma^2}\right). \quad (42)$$

The simulation uses

$$\sigma = 0.5.$$

For zero-mean noise,

$$\mathbb{E}[z_n] = d_{s,j,k}^{\text{true}}, \quad \text{Var}(z_n) = \sigma^2. \quad (43)$$

The measurement is therefore unbiased in expectation, although individual samples may lie above or below the true distance. Since the initial sensor range is also 0.5, the selected standard deviation is large relative to the physical measurement interval and makes filtering particularly important.

2.17 Exponential Moving-Average Filter

For every valid geometric intersection, the simulation generates a batch of $N = 1000$ noisy measurements of the same ideal distance. The first-order exponential moving-average estimate is initialized by

$$\hat{d}_0 = z_0 \quad (44)$$

and updated for $n = 1, \dots, 999$ according to

$$\hat{d}_n = \alpha z_n + (1 - \alpha) \hat{d}_{n-1}. \quad (45)$$

The implemented coefficient is

$$\alpha = 0.01.$$

The final value used by the obstacle-detection logic is

$$d_{s,j,k}^{\text{meas}} = \hat{d}_{999}. \quad (46)$$

The hit point is then recalculated from the filtered distance:

$$\mathbf{p}_{s,j,k}^{\text{hit}} = \mathbf{p}_k + d_{s,j,k}^{\text{meas}} \mathbf{u}_{s,k}. \quad (47)$$

After filtering, the measurement must again satisfy

$$0 \leq d_{s,j,k}^{\text{meas}} \leq R_s. \quad (48)$$

Consequently, a geometric intersection can be rejected if the final filtered measurement lies outside the valid range.

2.18 Influence of the Filter Coefficient

The coefficient α determines the balance between the new noisy measurement and the previous filtered estimate. When α is small, the filter gives most of the weight to its previous output:

$$\alpha \rightarrow 0 \quad \Rightarrow \quad \hat{d}_n \approx \hat{d}_{n-1}.$$

This produces strong smoothing because rapid variations in the noisy measurements have only a small influence on the output. However, the filtered estimate responds more slowly when the measured distance changes.

When α approaches one,

$$\alpha \rightarrow 1 \quad \Rightarrow \quad \hat{d}_n \approx z_n,$$

the current measurement has a greater influence on the output. The filter therefore reacts more quickly, but a larger part of the sensor noise remains in the filtered signal. At the limiting value $\alpha = 1$, no smoothing is performed.

Repeated substitution of Equation (45) gives

$$\hat{d}_n = (1 - \alpha)^n z_0 + \alpha \sum_{j=1}^n (1 - \alpha)^{n-j} z_j. \quad (49)$$

Older measurements therefore receive exponentially decreasing weights. This allows the filter to maintain a memory of previous values without storing the complete measurement history during the recursive calculation.

The influence of α was evaluated using two values, $\alpha = 0.01$ and $\alpha = 0.6$, while keeping the noise standard deviation constant at

$$\sigma = 0.5.$$

For $\alpha = 0.01$, the recursive equation becomes

$$\hat{d}_n = 0.01 z_n + 0.99 \hat{d}_{n-1}.$$

Only one percent of the new measurement is introduced at each iteration, while the previous filtered estimate contributes ninety-nine percent. Consequently, the output is highly smoothed, but several measurements are required before it adapts to a change in the measured distance.

For $\alpha = 0.6$, the filter becomes

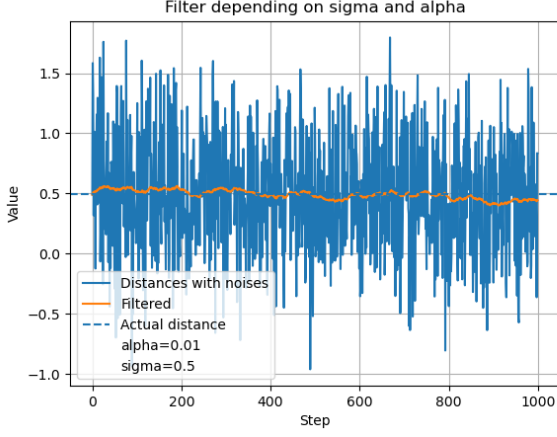
$$\hat{d}_n = 0.6 z_n + 0.4 \hat{d}_{n-1}.$$

In this case, the current measurement contributes sixty percent of the new estimate. The filtered output therefore follows changes much more rapidly, although the fluctuations produced by the Gaussian noise are less attenuated.

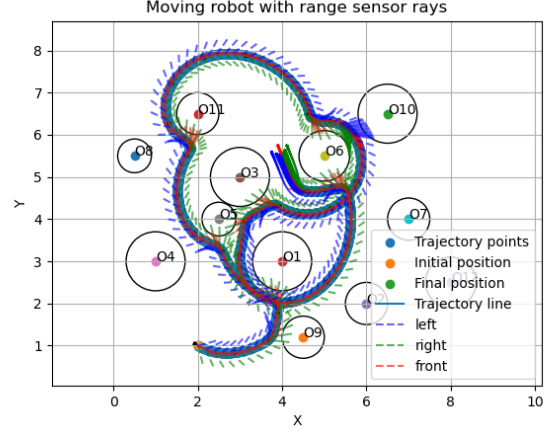
Figure 1 shows the effect of the filter coefficient on the measurements generated for the first obstacle detection. With $\alpha = 0.01$, the filtered signal remains considerably smoother than the noisy measurements. The strong contribution of the previous estimate prevents individual noisy samples from causing large changes in the filter output.

With $\alpha = 0.6$, the filtered signal follows the noisy measurements more closely. This allows it to adapt more rapidly, but the output contains noticeably larger fluctuations. Therefore, increasing α improves the response speed at the cost of weaker noise attenuation.

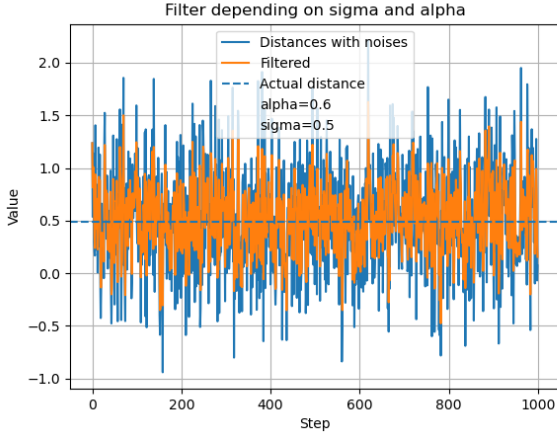
The trajectory plots show the combined result of goal-seeking control, range sensing, filtered obstacle detection, and reactive avoidance. However, the simulated sensor noise is generated randomly. Consequently, differences between individual trajectories are influenced both by the selected value of α and by the particular noise samples generated during each simulation run.



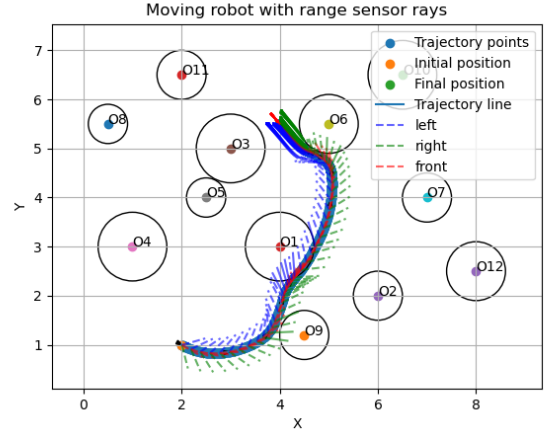
(a) Noisy and filtered sensor measurements for $\alpha = 0.01$ and $\sigma = 0.5$.



(b) Robot trajectory obtained using $\alpha = 0.01$.



(c) Noisy and filtered sensor measurements for $\alpha = 0.6$ and $\sigma = 0.5$.



(d) Robot trajectory obtained using $\alpha = 0.6$.

Figure 1: Comparison of the sensor filtering results and the corresponding robot trajectories for two exponential-filter coefficients. A small value of α produces stronger smoothing but a slower response, whereas a larger value produces a faster response but retains more measurement noise.

2.19 Filter Transfer Function and Noise Reduction

Ignoring initial conditions, the filter relation can be written in the z domain as

$$H(z) = \frac{\hat{D}(z)}{Z(z)} = \frac{\alpha}{1 - (1 - \alpha)z^{-1}}. \quad (50)$$

The pole is

$$z_p = 1 - \alpha. \quad (51)$$

For $0 < \alpha \leq 1$, the pole lies inside the unit circle and the filter is stable. Under the assumptions of a constant true distance and independent white Gaussian noise, the steady-state output variance is

$$\text{Var}(\hat{d}) = \frac{\alpha}{2 - \alpha} \sigma^2. \quad (52)$$

For $\alpha = 0.01$, the theoretical variance factor is

$$\frac{0.01}{2 - 0.01} \approx 0.00503.$$

Thus, the selected coefficient strongly reduces the noise variance for a constant distance, at the cost of a slower transient response.

3 Integrated Method

3.1 Simulation Configuration

The main simulation parameters are summarized in Table 2. The environment contains twelve circular obstacles with individually specified centers and radii.

3.2 Sensing–Decision–Motion Cycle

At every simulation step, the method performs the following sequence:

Table 2: Main Simulation Parameters.

Parameter	Value
Initial robot position	$[2.0, 1.0]^T$
Goal position	$[4.0, 5.5]^T$
Initial orientation	-30°
Time step Δt	0.01
Maximum simulation steps	10000
Goal tolerance	0.01
Initial sensor range	0.5
Near-goal sensor range	0.3
Angular gain K_θ	0.5
Initial linear gain K_v	0.5
Near-goal linear gain	0.1
Maximum linear speed	1.0
Noise standard deviation	$\sigma = 0.5$
Filter coefficient	$\alpha = 0.01$
Samples per filtered hit	1000

1. Calculate the current distance to the goal and stop if the tolerance has been reached.
2. Apply the near-goal parameter adjustment when $d_k < 1$.
3. Construct the left, right, and front finite sensor rays from the current robot pose.
4. Test every sensor ray against every circular obstacle using projection, perpendicular distance, and ray-circle intersection.
5. For each valid geometric hit, generate a batch of 1000 noisy Gaussian measurements and apply the exponential moving-average filter.
6. Reject filtered distances outside the finite sensor range and retain the closest valid obstacle for each sensor.
7. Convert the three sensor outputs into the binary variables D_L , D_R , and D_F .
8. If any sensor detects an obstacle, apply the first matching reactive rule. Otherwise, calculate v_k and ω_k with the goal-seeking controllers.
9. Update the robot orientation, recompute its heading, and advance the position using the discrete motion model.
10. Store the trajectory, orientation, speed, angular error, distance to the goal, sensor endpoints, and current detections.

This order creates the complete information flow

geometry $\rightarrow$ sensing $\rightarrow$ noise
 $\rightarrow$ filtering $\rightarrow$ detection
 $\rightarrow$ behavior selection $\rightarrow$ motion.

3.3 Recorded Data and Visual Interpretation

The simulation stores the robot trajectory and the values of θ_k , v_k , $e_{\theta,k}$, and d_k over time. It also stores the

three sensor endpoints and the selected detections at every step. These histories make it possible to visualize the robot path, circular obstacles, motion direction, and sensor rays. A sensor ray is represented as active when it contains a detection and inactive when no valid obstacle is returned.

For the first geometric detection only, the simulation also stores all 1000 noisy samples and their corresponding filtered values. The exact geometric hit distance is retained as a reference. These three signals can later be plotted together to study how the parameters σ and α affect measurement dispersion, smoothing, and filter response.

4 Implementation-Specific Interpretation

4.1 Batch Filtering Rather Than Persistent Online Filtering

The filter is restarted for every valid geometric ray-circle intersection. It processes 1000 independently generated measurements of one fixed geometric distance and returns the final estimate $\hat{d}_{999}$. The previous filtered value is not carried from robot step k to step $k+1$. Therefore, the implemented method is best interpreted as repeated-measurement batch smoothing at each candidate hit, not as one persistent online filter operating continuously over the complete robot trajectory.

The stored noisy and filtered lists are limited to the first valid detection for visualization, but the generation and filtering procedure is applied again to every valid candidate intersection used during the simulation.

4.2 Measurement Naming and Validation

After Gaussian noise is added, the resulting quantity is a measured distance rather than a true distance. Mathematically, the relation is $z_n = d^{\text{true}} + \eta_n$, regardless of the internal variable name used by the implementation. The final filtered value replaces the ideal geometric distance before the hit point is calculated and before the obstacle is accepted as a valid detection.

4.3 Inactive Escape Mode

The source file contains a proposed loop-detection and escape-mode strategy, but this block is inactive. It therefore does not form part of the implemented navigation method described in this paper. The active controller consists only of the proportional goal-seeking behavior, the priority-based reactive rules, and the near-goal parameter adjustment. Loop detection may be considered a future extension for situations in which purely reactive avoidance produces oscillations or local cycles.

5 Project Task

Create a two-dimensional mobile robot simulation with one goal, three finite range sensors, and circular obstacles. Use normalized angular error for proportional steering and a saturated distance-dependent law for linear speed. Detect obstacles by projecting each circle center onto every sensor ray, calculating the perpendicular distance, and obtaining the nearest valid ray-circle intersection. Convert the left, right, and front detections into binary states and give reactive avoidance priority over free-space goal control. Add zero-mean Gaussian noise to every ideal hit distance, smooth 1000 repeated measurements with an exponential moving-average filter, and use the final estimate for range validation and hit-point calculation. Store the trajectory and the first noisy-filtered sequence for later visualization.